

Compression d'image par DCT et stockage CSR

Portage en C++ du projet Python de MAM3

Romain Ben Karim Zrig

MAM4 Polytech Nice Sophia | Projet C++

16 septembre 2026

Point de départ : le projet Python de MAM3

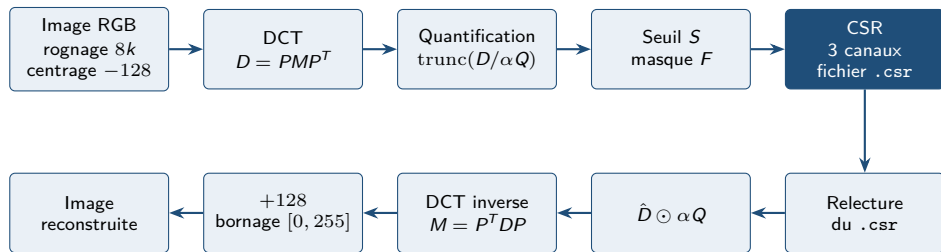
Version Python (janvier 2026)

- ▶ `jpeg_compression.py` : DCT, quantification, troncature
- ▶ `scipy.sparse.csr_matrix` pour le stockage creux
- ▶ application Streamlit : seuil réglable, métriques, export `.npz`
- ▶ rapport : facteur α , matrices Q , bruit poivre et sel

Objectif du projet C++

- ▶ tout refaire en C++20, sans numpy ni scipy
- ▶ appliquer les notions du cours : invariants, RAll, règle de zéro, interfaces
- ▶ retrouver les mêmes coefficients que Python
- ▶ mesurer le gain de vitesse

L'algorithme, commun aux deux versions



$P_{k,i} = \frac{C_k}{2} \cos \frac{(2i+1)k\pi}{16}$ est orthogonale : l'inverse de la DCT est sa transposée.

Masque carré (version Python) : $k, l < F$ Masque triangulaire (sujet) : $k + l < F$

Organisation du dépôt

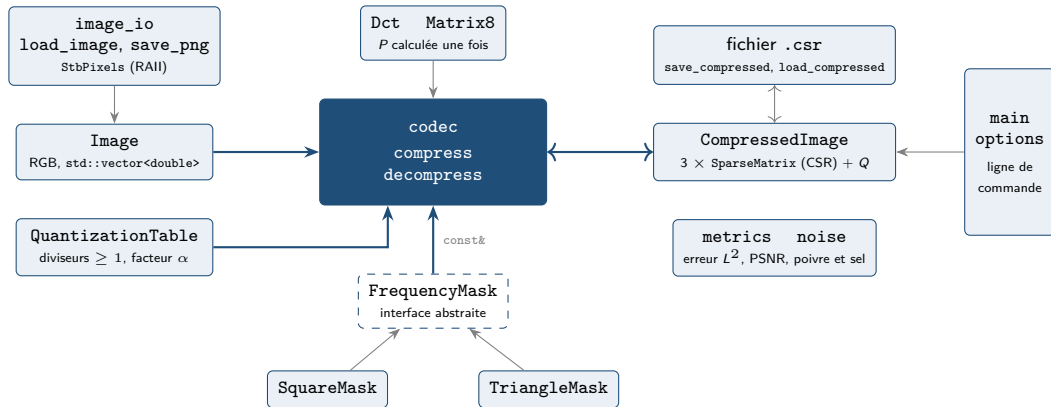
```
jpeg-compression/  
|-- README.md  
|-- python/          projet MAM3  
|-- cpp/  
|   |-- Makefile  
|   |-- include/      en-têtes  
|   |-- src/          définitions  
|   |-- tests/        tests unitaires  
|   |-- scripts/      comparaison, figures  
|   |-- images/       images de test  
|   |-- third_party/  stb  
|   `-- docs/         rapport, slides  
`-- .github/workflows/ CI
```

Construire et lancer

```
make           # ./jpeg_csr  
make test      # tests unitaires  
make demo      # plusieurs réglages  
make test SANITIZE=1
```

- ▶ compilation séparée .hpp / .cpp
- ▶ -std=c++20 -Wall -Wextra -pedantic
- ▶ CI GitHub : tests sous AddressSanitizer et UBSan

Architecture C++



Les notions du cours dans le code

Notion	Où elle apparaît
<code>const T& et T&</code>	<code>compress(const Image&) lit, add_salt_and_pepper(Image&)</code> modifie
Classe, invariant, private	<code>Image, QuantizationTable, SparseMatrix, masques</code>
Constructeur, exceptions	invariant vérifié, sinon <code>std::invalid_argument</code>
RAII, copie interdite	<code>StbPixels</code> libère le tableau de <code>stb</code> ; <code>std::ofstream</code> ferme le fichier
Règle de zéro	<code>std::vector</code> et <code>std::array</code> partout : aucun destructeur à écrire
Déplacement	constructeur de <code>SparseMatrix</code> : paramètres par valeur puis <code>std::move</code>
Surcharge d'opérateurs	<code>Matrix8::operator()</code> (const et non const), <code>operator*</code>
Surcharge de fonctions	<code>write</code> et <code>read</code> pour <code>uint32</code> , <code>double</code> et vecteurs
Interface abstraite	<code>FrequencyMask</code> : <code>virtual</code> , <code>override</code> , destructeur virtuel
Compilation séparée	<code>include/</code> , <code>src/</code> , <code>Makefile</code> , <code>-isystem</code> pour <code>stb</code>

Invariants : rendre explicites les hypothèses du Python

```
// Invariant : tous les diviseurs sont >= 1.  
// Comme |D| <= 8 * 128 = 1024, chaque coefficient  
// quantifié tient dans un std::int16_t.  
class QuantizationTable {  
public:  
    // lance std::invalid_argument si un diviseur < 1  
    explicit QuantizationTable(const Matrix8& divisors);  
  
    static QuantizationTable standard();  
    static QuantizationTable uniform(double value);  
    QuantizationTable scaled(double alpha) const;  
  
    double operator()(std::size_t k, std::size_t l) const;  
  
private:  
    Matrix8 divisors_;  
};
```

- ▶ Image : dimensions non nulles, taille = $w \times h \times 3$
- ▶ SparseMatrix : pointeurs croissants, colonnes triées, aucun zéro stocké
- ▶ CompressedImage : multiples de 8, exactement 3 canaux
- ▶ masques : $1 \leq F \leq 8$ (carré), $1 \leq F \leq 15$ (triangle)

À la relecture d'un .csr, les constructeurs revérifient tout : un fichier corrompu est rejeté au lieu de produire une image fausse.

Polymorphisme : le masque est choisi à l'exécution

```
class FrequencyMask {
public:
    virtual bool keeps(std::size_t k,
                       std::size_t l) const = 0;
    virtual std::string name() const = 0;
    virtual ~FrequencyMask() = default;
};

// k < F et l < F    (version Python)
class SquareMask : public FrequencyMask { ... };

// k + l < F         (sujet MAM3)
class TriangleMask : public FrequencyMask { ... };
```

```
CompressedImage compress(
    const Image& image,
    const QuantizationTable& table,
    int threshold,
    const FrequencyMask& mask);

int run_compress(const Options& options)
{
    if (options.mask == MaskKind::triangle) {
        const TriangleMask mask{options.cutoff};
        return compress_with_mask(options, mask);
    }
    const SquareMask mask{options.cutoff};
    return compress_with_mask(options, mask);
}
```

Le masque vit sur la pile et passe par référence de base : pas de new ni de delete, et ajouter un masque ne modifie pas compress.

Ressources : RAII autour de stb, règle de zéro ailleurs

```
// Tableau alloué par stbi_load : libéré par le  
// destructeur, même si une exception survient.  
class StbPixels {  
public:  
    explicit StbPixels(const std::string& path);  
    ~StbPixels();    // stbi_image_free(data_)  
  
    StbPixels(const StbPixels&) = delete;  
    StbPixels& operator=(const StbPixels&) = delete;  
  
private:  
    int width_{};  
    int height_{};  
    unsigned char* data_{nullptr};  
};
```

```
// CSR : trois std::vector, aucune opération  
// spéciale écrite (règle de zéro).  
class SparseMatrix {  
    ...  
private:  
    std::size_t rows_;  
    std::size_t cols_;  
    std::vector<std::int16_t> values_;  
    std::vector<std::int32_t> column_indices_;  
    std::vector<std::int32_t> row_pointers_;  
};
```

Fichier .csr : signature, dimensions, Q , puis pour chaque canal nnz, pointeurs de lignes, colonnes et valeurs.

Utilisation

```
./jpeg_csr compress images/astronaut.png --alpha 5 --out resultats/alpha5  
./jpeg_csr decompress resultats/alpha5/astronaut.csr relue.png
```

```
Image           : images/astronaut.png (512 x 512, rognée à 512 x 512)  
Réglages        : Q standard, alpha 1.00, seuil 2, masque carré F = 6  
Coefficients    : 45566 non nuls sur 786432 (taux de conservation 5.79 %)  
Qualité         : erreur L2 relative 5.43 %, PSNR 30.49 dB  
Mémoire dense   : 6144.00 Ko (float64, comme img.nbytes)  
Mémoire CSR     : 273.00 Ko (valeurs int16, indices int32)  
Gain mémoire    : dense / CSR = 22.51x  
Fichier .csr    : 273.52 Ko, image source 773.00 Ko  
Temps           : compression 28.23 ms, décompression 21.55 ms
```

Options : `--table standard|uniform|low|high`, `--alpha`, `--threshold`,
`--mask square|triangle`, `--cutoff`, `--noise`, `--out`.

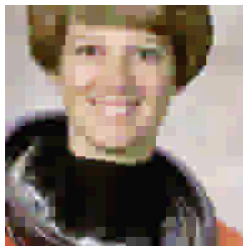
Résultats : le facteur de qualité α



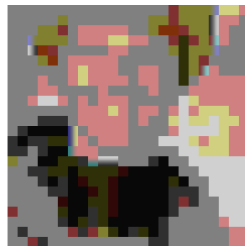
originale



$\alpha = 1$



$\alpha = 5$



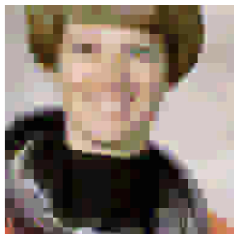
$\alpha = 20$

	Conservation	Erreur L^2	Taille CSR	Gain dense/CSR
$\alpha = 1$	5,79 %	5,43 %	273 Ko	22,5
$\alpha = 5$	2,15 %	11,29 %	105 Ko	58,5
$\alpha = 20$	0,64 %	32,95 %	35 Ko	174,0

Résultats : matrices de quantification



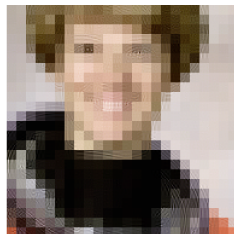
Q standard



Q uniforme (100)



basses fréquences



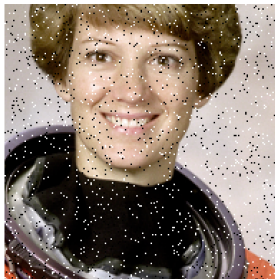
hautes fréquences

	Conservation	Erreur L^2	Taille CSR
Q standard (psychovisuelle)	5,79 %	5,43 %	273 Ko
Q uniforme	1,65 %	13,85 %	82 Ko
diviseurs faibles en basses fréquences	8,76 %	6,52 %	409 Ko
diviseurs faibles en hautes fréquences	25,45 %	17,49 %	1 179 Ko

Résultats : bruit poivre et sel



image pure



bruitée à 5 %



reconstruite, masque triangulaire $F = 4$

Erreur L^2 par rapport à l'image pure :

- ▶ image bruitée : 24,0 %
- ▶ reconstruite, triangle $F = 4$: 11,3 %
- ▶ reconstruite, réglages Python : 12,8 %

Le bruit est fait de variations très rapides, donc de hautes fréquences : la troncature agit comme un filtre passe-bas, au prix d'un flou des contours.

Validation et performances

17

fonctions de test

- ▶ tests pytest de la version Python
- ▶ invariants de chaque classe
- ▶ aller-retour par fichier `.csr`
- ▶ ligne de commande

25

coefficients différents sur 786 432

Même nombre de non nuls (45 566). En Python, D/Q vaut 23,999 999 999 999 996 au lieu de 24 (image passée par $/255$ puis $\times 255$) : la troncature donne 23.

× 11

plus rapide que Python

Compression et décompression de 512×512 : 50 ms en C++ contre 543 ms en Python (même machine, meilleur de 3 essais).

Bilan et perspectives

Bilan

- ▶ toutes les fonctionnalités Python reproduites en C++20
- ▶ mêmes coefficients, onze fois plus vite
- ▶ invariants : dimensions, type `int16`, fichier CSR cohérent
- ▶ aucune gestion mémoire manuelle hors `StbPixels`

Perspectives

- ▶ parcours zigzag et codage entropique (RLE, Huffman) : vrai fichier JPEG
- ▶ espace YCbCr et sous-échantillonnage de la chrominance
- ▶ plusieurs masques polymorphes avec `std::unique_ptr`

<https://github.com/BnRomain/jpeg-compression>